THE NORTHCAP UNIVERSITY

Department of Computer Science & Engineering

Machine Learning-Based Network Intrusion Detection System

End-Term Project Report | CSL236 — Introduction to AI and Machine Learning

Team member	Roll number
Cheshna Mehra	24csu044
Faguni	24csu290
Riya Fulambarkar	24csu174
Section	[Add section]
Dataset	CICIDS2017 — MachineLearningCVE

Submission note

This report is a project-specific submission draft aligned to the CSL236 End-Term Project Guidelines. The training code, API, tests, Docker and CI workflow are included in the repository. Dataset-dependent MLflow metrics, screenshots, GitHub URL and live deployment URL must be added after the team runs and publishes the project. No run, screenshot or deployment is represented as completed until verified.

1. Problem statement and dataset

Network intrusion detection is the task of identifying network flows that may represent malicious activity. This project uses supervised binary classification to label each flow as BENIGN (0) or ATTACK (1). The service is designed as decision support: a positive prediction should be reviewed alongside firewall, IDS and endpoint telemetry rather than automatically blocking traffic.

Dataset source: CICIDS2017 MachineLearningCVE flow CSVs published by the Canadian Institute for Cybersecurity: https://www.unb.ca/cic/datasets/ids-2017.html. The project reuses the same network-flow dataset family as the team's supplied notebook. CSV files are placed in data/ and are not committed when larger than 50 MB. The training script reads files in chunks, samples up to 10,000 rows per original label, removes duplicate rows and maps BENIGN to 0 and all other labels to 1.

The final data contract is generated from the training partition and written into README.md and DATA_CONTRACT.md by scripts/export_contract.py. It lists every input feature, its numeric type, training-derived minimum/maximum and rationale. The API uses the same contract for Pydantic validation.

EDA highlights to record after the real-data run

Insert the actual dataset shape, class counts, missing/infinite-value summary and feature distribution plots from the training run. The supplied preprocessing notebook already contains EDA and preprocessing exploration; it is retained under notebooks/ as supporting project work.

[EVIDENCE PLACEHOLDER: dataset class distribution and missing-value/EDA screenshot]

2. Preprocessing and modelling

The pipeline uses a stratified 80:20 train/test split before fitting any learned preprocessing. A DataFrame-preserving median imputer handles missing values after infinities are converted to missing. NIDSFeatureEngineer learns strongly skewed non-negative columns from training data, applies log1p to those columns, and derives ordinal Flow Duration quartile bins when the column is available. StandardScaler, VarianceThreshold and PCA are fitted only on the training partition; PCA retains 95% of explained variance.

Three MLflow-tracked configurations are trained: Logistic Regression, Random Forest and Extra Trees. The model selection criterion is test F1-score, with accuracy as a tie-breaker. The final pipeline is serialized to model/model.joblib; model/metadata.json stores the feature contract, selected algorithm, metrics and representative domain-logic test examples.

Experiment comparison table — populate from training_results.json

Run	Algorithm	Configuration	Accuracy	F1
1	Logistic Regression	C = CLI value	[run]	[run]
2	Random Forest	n_estimators, max_depth	[run]	[run]
3	Extra Trees	n_estimators, max_depth	[run]	[run]

Do not enter assumed metrics. Run `python train.py` on the actual CICIDS dataset and copy the values from training_results.json and the MLflow UI.

3. System architecture

The project follows the workshop's end-to-end MLOps reference pattern. Dataset CSVs flow into the training script; the script runs multiple model configurations and logs parameters/metrics to MLflow's SQLite backend. The best fitted pipeline is serialized once. FastAPI loads it at application startup and validates every feature against the generated Pydantic contract. The HTML UI sends predictions to the API. Docker packages the serving-only runtime, while GitHub Actions runs training and pytest on pushes/pull requests. Render can host the container. Prediction inputs, predictions and latency are written to a JSONL log for drift review.

DATASET CSVs	**train.py + preprocessing**	**MLflow SQLite**
↓	↓	↓
Feature contract	**Best model.joblib**	**Experiment comparison**
↓	↓	↓
FastAPI + Pydantic	**Docker image**	**GitHub Actions CI**
↓	↓	↓
HTML UI + /predict	**Render deployment**	**pytest + logs**

Technology stack: Python 3.11, pandas, NumPy, scikit-learn, MLflow, FastAPI, Pydantic, pytest, Docker, GitHub Actions and Render.

4. API, validation and user interface

The FastAPI service exposes GET / for the browser UI, GET /health for health checking, POST /predict for inference and the automatically generated /docs OpenAPI page. The model is loaded once when the application process starts, not inside the request handler. The prediction request contains all numeric input features. Pydantic creates a constrained field for each feature using the training-derived lower and upper bounds; missing fields, unknown fields, non-numeric values, NaN/Infinity and out-of-range values are rejected with HTTP 422.

The UI is generated from the API's OpenAPI schema so that the input fields and limits follow the model contract. It displays the predicted class, attack probability and up to three rule-based tips. Tips encourage investigation and correlation with other security telemetry; they are not presented as an automatic remediation policy.

Evidence to add

[SCREENSHOT PLACEHOLDER: home page with a valid sample prediction]

[SCREENSHOT PLACEHOLDER: /docs showing PredictionRequest fields]

[SCREENSHOT PLACEHOLDER: invalid out-of-range request returning HTTP 422]

5. Automated testing and CI

The pytest suite includes service health, UI loading, valid prediction output, a domain-logic comparison, out-of-range validation and missing-feature validation. The domain-logic test compares a correctly classified benign-like holdout sample with an attack-like holdout sample saved in metadata and checks that the attack-like example receives a higher attack probability. This test is selected from the trained model's holdout predictions and is intended as a regression sanity check, not a universal security rule.

GitHub Actions runs on pushes and pull requests to main. It installs the pinned training environment, generates a documented deterministic synthetic fixture for CI smoke-testing, trains three MLflow-tracked configurations and executes pytest. The synthetic fixture is strictly for pipeline CI; final model metrics and deployment must use the public CICIDS dataset. This avoids committing a large dataset to Git.

Evidence to add

[SCREENSHOT PLACEHOLDER: `pytest -v --tb=short` with all tests passing]

[SCREENSHOT PLACEHOLDER: latest green GitHub Actions workflow run]

[SCREENSHOT PLACEHOLDER: MLflow UI showing at least three runs and metric comparison]

6. Docker, deployment and logging

The Dockerfile uses python:3.11-slim and installs only requirements-serve.txt, excluding MLflow and pytest from the runtime image. Build after training has generated model/model.joblib and model/metadata.json. The container serves the API on port 8000. Render is configured through render.yaml with /health as the health-check path; the service should be configured to restart after failure and kept available through the evaluation/demo period.

```
python train.py --n_estimators 100 --max_depth 12
pytest -v --tb=short
docker build -t team10-nids .
docker run -d -p 8000:8000 --restart unless-stopped team10-nids
uvicorn app:app --reload --port 8000
```

Every successful prediction appends timestamp, input features, predicted class, attack probability and latency to logs/predictions.jsonl. The log is excluded from Git. It can be compared with training statistics to detect drift; the current project includes input logging but does not claim a production drift alarm.

Deployment evidence and URLs

GitHub repository: [ADD PUBLIC/INSTRUCTOR-SHARED REPOSITORY URL]
Live service: [ADD VERIFIED RENDER/RAILWAY URL]
Swagger docs: [LIVE URL]/docs
Docker image size: [RECORD `docker images team10-nids` OUTPUT]

7. Limitations, future work and contributions

The binary target merges different attack families, so the model cannot identify the exact attack type. Training-derived min/max input bounds can reject legitimate traffic values outside the training range; the contract should be reviewed when new traffic populations are introduced. Class imbalance and temporal drift may affect performance. A prediction is not a substitute for security analyst review.

Future work includes multiclass attack-family prediction, probability-threshold calibration, SHAP or feature-importance explanations, a scheduled drift report comparing logged input means/standard deviations with training statistics, and automated CD after CI passes.

Individual contribution — confirm with the team and Git commit history

Member	Roll number	Actual contribution (complete before submission)
Cheshna Mehra	24csu044	[Add verified tasks and commits]
Faguni	24csu290	[Add verified tasks and commits]
Riya Fulambarkar	24csu174	[Add verified tasks and commits]

References

1. CSL236 End-Term Project Submission Guidelines, The NorthCap University (6 pages).
2. BrainMentors, End-to-End MLOps: Production Deployment workshop material.
3. Canadian Institute for Cybersecurity, CICIDS2017 dataset: https://www.unb.ca/cic/datasets/ids-2017.html.
4. Team 10, supplied NIDS preprocessing and modelling notebook.

8. Workshop integration and operational evidence

The team attended the four-hour BrainMentors workshop, “End-to-End MLOps: Production Deployment”. Its stated toolchain includes Python 3.11, model-serving APIs, Docker and Docker Compose, MLflow Tracking and Registry, GitHub Actions, Docker Hub, Render and Prometheus instrumentation. The curriculum progresses from model interfacing and containerization to experiment/model versioning, cloud deployment, observability, and CI/CD.

Workshop concept	How it is reflected in this project
API-based serving	FastAPI exposes /, /health, /predict and /docs; Pydantic enforces the generated feature contract.
Artifact lifecycle	Joblib stores the complete fitted preprocessing + classifier pipeline. app.py loads it once at process startup.
Experiment tracking	Three model configurations log parameters, metrics and model artifacts to MLflow with a SQLite backend.
Model registry	The winning run is registered as Team10NIDS with the champion alias when MLflow Registry is available.
Container portability	Dockerfile uses python:3.11-slim and installs requirements-serve.txt only.
CI automation	GitHub Actions trains and tests on pushes/pull requests to main.
Cloud hosting	render.yaml provides a Render Docker-service starter; live URL remains pending team account configuration.
Observability	Prometheus prediction counters, error counter and latency histogram are exposed at /metrics; JSONL prediction inp

Current validation status

The source files pass Python syntax compilation. A disposable, deterministic synthetic CI fixture was used to smoke-test the training pipeline and FastAPI contract; all eight pytest tests passed in that local smoke run. The execution environment did not have MLflow installed, so MLflow calls were shimmed for this local smoke test. The real MLflow SQLite tracking/Registry run must be verified after installing requirements.txt in the team's Python 3.11 environment. No CICIDS dataset training, Docker build, GitHub Actions run or live deployment is claimed as completed.

Required evidence before final submission: MLflow UI screenshot showing 3+ actual CICIDS runs; UI and /docs screenshots; valid prediction and 422 response; pytest output; green GitHub Actions run; Docker image size; live Render URL; and a sample logs/predictions.jsonl file. Insert only genuine screenshots and verified URLs.